Contrato de Servicio: Distributed Storage Service

1. Información general

Campo	Valor
Nombre del servicio	Distributed Storage Service
Descripción	Servicio central de almacenamiento distribuido: gestiona el Repository físico, metadatos, versionado, cuotas y replicación de todo el contenido de la plataforma.
Objetivo	Ofrecer un punto único y consistente de acceso a archivos, con garantías de integridad, versionado y control de cuota, independientemente de qué servicio de negocio lo consuma.
Responsabilidad	Almacenamiento físico distribuido, repositorio Home por usuario, replicación, versionado, cuotas, organización jerárquica de archivos/directorios, gestión de metadatos.
Tecnología	A definir por el equipo (no impuesta por el enunciado); se recomienda un lenguaje con buen soporte de I/O concurrente. El contrato es independiente de la tecnología.
Protocolo de comunicación	HTTP/REST interno para operaciones de archivo; puede exponer una interfaz adicional binaria para transferencias grandes (fuera del alcance de este contrato funcional).
Servicios consumidores	Shared File Service, File Sync Service, Photo Album Service, Streaming Service, HPC Service (almacenamiento de resultados), Monitoring Service (métricas de capacidad).
Servicios proveedores	Authentication Service (`ValidarToken`, `ObtenerHomeUsuario`).

2. Funcionalidades (operaciones expuestas)

1. SubirArchivo
2. DescargarArchivo
3. EliminarArchivo
4. ListarArchivos
5. CrearDirectorio

6. EliminarDirectorio
7. MoverArchivo / RenombrarArchivo
8. ObtenerMetadatos
9. ActualizarMetadatos
10. ListarVersiones
11. RestaurarVersion
12. ObtenerCuotaUsuario
13. ActualizarCuota (administrativo)
14. VerificarIntegridad
15. CrearHomeUsuario (interno, disparado por evento de Authentication)
16. BuscarArchivos
17. ObtenerEstadisticasUso (para Monitoring)
18. ReplicarArchivo (interno, entre nodos de almacenamiento)

---

3. Contrato de cada operación

3.1 SubirArchivo

Nombre: `SubirArchivo` **Descripción:** Almacena un nuevo archivo o una nueva versión de un archivo existente dentro del árbol lógico del usuario. **Protocolo:** HTTP/REST (multipart o streaming, según tamaño)

Parámetros de entrada:

Nombre	Tipo	Obligatorio	Descripción	Restricciones/Validaciones
`token`	string	Sí	Token de sesión válido	Validado contra Authentication Service
`rutaDestino`	string	Sí	Ruta lógica donde se guarda (relativa al Home o a un directorio compartido)	No puede contener `..` (traversal), longitud máxima 1024 caracteres
`nombreArchivo`	string	Sí	Nombre del archivo	1-255 caracteres, caracteres válidos de sistema de archivos
`contenido`	binary/stream	Sí	Bytes del archivo	Tamaño máximo configurable por política de cuota
`checksum`	string	No	Hash (ej. SHA-256) provisto por el cliente para verificar integridad al llegar	Si se provee, se valida contra el checksum calculado en el servidor
`sobrescribir`	boolean	No	Si es `false` y el archivo existe, se crea una nueva versión en lugar de sobrescribir	Por defecto `false`

Datos de salida:

Nombre	Tipo	Descripción
`archivoId`	UUID	Identificador único del archivo
`version`	int	Número de versión resultante
`rutaFinal`	string	Ruta lógica final almacenada
`checksumServidor`	string	Hash calculado por el servidor
`tamanoBytes`	long	Tamaño almacenado
`fechaSubida`	datetime	Timestamp de la operación

Posibles errores:

Código	Descripción	Posible solución
`STG-001`	Cuota de usuario excedida	Liberar espacio o solicitar ampliación de cuota
`STG-002`	Ruta de destino inválida (traversal o permisos)	Corregir ruta
`STG-003`	Checksum no coincide (corrupción en tránsito)	Reintentar subida
`STG-004`	Permiso insuficiente sobre el directorio destino	Verificar permisos estilo Unix
`STG-005`	Espacio insuficiente en el nodo de almacenamiento	Redirigir a otro nodo / alertar a Monitoring
`STG-006`	Token inválido/expirado	Reautenticar

Reglas de negocio:

- Toda subida debe validarse contra la cuota **antes** de escribir en disco, no después (evita escrituras parciales que luego hay que revertir).
- Si `sobrescribir=false` y el archivo ya existe, se crea automáticamente una nueva versión (no un error) — el versionado es transparente por defecto.
- Toda escritura exitosa dispara replicación asíncrona hacia al menos un nodo secundario (ver operación interna `ReplicarArchivo`).

Seguridad: Requiere token válido. Requiere permiso de escritura (`archivos:escribir`) sobre la ruta destino, evaluado con semántica estilo Unix (propietario/grupo/otros).

Dependencias: Authentication Service (`ValidarToken` para autenticar, y verificación de permisos propios del árbol de directorios que este servicio mantiene).

Flujo interno:

1. Presentación recibe la solicitud multipart/stream.
2. Aplicación valida el token contra Authentication Service.
3. Dominio verifica permisos sobre la ruta y calcula si la cuota lo permite.
4. Persistencia escribe el archivo físico y calcula checksum.
5. Dominio registra metadatos (versión, tamaño, fecha, propietario).
6. Se emite evento `ArchivoSubido`.
7. Se encola la replicación asíncrona hacia nodo(s) secundario(s).
8. Presentación responde con metadatos de la operación.

3.2 DescargarArchivo

Protocolo: HTTP/REST (streaming)

Parámetros de entrada: `token`, `archivoId` o `rutaArchivo`, `version` (opcional, por defecto la última).

Datos de salida: stream de bytes + metadatos (`nombreArchivo`, `tamanoBytes`, `checksum`, `version`, `tipoMime`).

Posibles errores:

Código	Descripción	Solución
`STG-101`	Archivo no encontrado	Verificar ruta/id
`STG-102`	Versión solicitada no existe	Listar versiones disponibles
`STG-103`	Permiso de lectura insuficiente	Solicitar acceso al propietario
`STG-104`	Nodo de almacenamiento con la réplica principal no disponible	Servir desde réplica secundaria automáticamente

Reglas de negocio: Si el nodo primario no responde, el servicio debe intentar servir desde una réplica antes de fallar — esta es la justificación de negocio de mantener replicación, no

solo redundancia "por si acaso".

Seguridad: Requiere token válido y permiso de lectura (`archivos:leer`) sobre el archivo.

Dependencias: Authentication Service.

Flujo interno: Presentación recibe solicitud → Aplicación valida token/permiso → Dominio localiza la réplica disponible más cercana/rápida → Persistencia stream de lectura → respuesta.

3.3 EliminarArchivo

Protocolo: HTTP/REST

Parámetros de entrada: `token`, `archivoId`, `eliminarTodasVersiones` (boolean, por defecto `false` — por defecto solo marca como eliminada la última versión, conservando historial).

Datos de salida: `confirmacion`.

Posibles errores: `STG-101` (no encontrado), `STG-103` (permiso insuficiente — requiere `archivos:eliminar`), `STG-201` (archivo en uso/bloqueado por otra operación, ej. sincronización en curso).

Reglas de negocio: Eliminación lógica (soft delete) por defecto, con purga física diferida (job de limpieza), para permitir recuperación ante error humano y para no romper una sincronización en curso desde otro cliente.

Seguridad: Requiere permiso `archivos:eliminar`.

Dependencias: Authentication Service.

Flujo interno: validar token/permiso → marcar versión(es) como eliminadas → emitir evento `ArchivoEliminado` → confirmar.

3.4 ListarArchivos / BuscarArchivos

Protocolo: HTTP/REST

Entrada: `token`, `rutaDirectorio` (para listar) o `criterioBusqueda` (nombre, tipo, rango de fecha, tamaño — para buscar), `paginacion` (offset/limit).

Salida: lista paginada de `MetadatoArchivo`.

Errores: `STG-103` (permiso insuficiente sobre el directorio), `STG-301` (criterio de búsqueda inválido).

Regla de negocio: el listado solo devuelve elementos sobre los que el usuario tiene al menos permiso de lectura — nunca se filtra en el cliente, siempre en el servidor.

Seguridad: Token válido; el filtrado de permisos es responsabilidad del servicio, no del consumidor.

Dependencias: Authentication Service.

3.5 CrearDirectorio / EliminarDirectorio / MoverArchivo

Mismo patrón de contrato que las anteriores. Puntos distintivos:

- `CrearDirectorio` hereda por defecto los permisos del directorio padre.
- `EliminarDirectorio` requiere que esté vacío o un flag explícito `recursivo=true`, y en ese caso elimina lógicamente todo su contenido.
- `MoverArchivo`/`RenombrarArchivo` debe preservar el historial de versiones y el `archivoId` (el id nunca cambia, solo la ruta).

Errores comunes adicionales: `STG-401` (directorio destino no existe), `STG-402` (directorio no vacío sin flag recursivo).

3.6 ListarVersiones / RestaurarVersion

Entrada (ListarVersiones): `token`, `archivoId`. **Salida:** lista de `Version` (número, fecha, tamaño, autor del cambio). **Entrada (RestaurarVersion):** `token`, `archivoId`, `versionObjetivo`. **Salida:** nueva versión creada como copia de la versión objetivo (nunca se "reescribe" el historial, restaurar crea una versión nueva). **Error específico:** `STG-102` (versión inexistente). **Regla de negocio:** restaurar nunca borra versiones intermedias — esto es clave para que File Sync Service pueda razonar sobre el historial sin ambigüedad.

3.7 ObtenerCuotaUsuario / ActualizarCuota

Entrada (ObtenerCuotaUsuario): `token`. **Salida:** `espacioUsado`, `espacioTotal`, `porcentajeUso`. **Entrada (ActualizarCuota, administrativo):** `token` de administrador, `usuarioId`, `nuevoLimite`. **Salida:** confirmación. **Error:** `AUTH-301`-equivalente (`STG-501`, permiso insuficiente — requiere rol `ADMIN`). **Regla de negocio:** el cálculo de espacio usado se hace sobre el tamaño de todas las versiones vivas (no eliminadas físicamente), lo que significa que el historial de versiones **cuenta contra la cuota** — decisión de diseño que debe comunicarse claramente a los usuarios.

3.8 CrearHomeUsuario (interno, disparado por evento)

Descripción: No se invoca directamente por un cliente; se dispara al consumir el evento `UsuarioRegistrado` emitido por Authentication Service. **Entrada:** `usuarioId`, `homePath` (provisto por Authentication). **Salida:** confirmación interna (no visible al cliente final). **Regla de negocio:** si la creación del Home falla, el servicio debe reintentar con backoff y alertar a Monitoring — un usuario sin Home no puede usar Shared File, Sync ni Photos.

3.9 VerificarIntegridad (interno/administrativo)

Entrada: `archivoId` o `rutaDirectorio` (para verificación masiva), `token` de administrador o proceso interno programado. **Salida:** reporte de archivos con checksum inconsistente entre réplicas. **Regla de negocio:** ejecución programada periódica, no solo a demanda — es la base para detectar corrupción silenciosa antes de que un usuario la note.

3.10 ReplicarArchivo (interno, entre nodos)

Descripción: Operación interna entre nodos del propio Storage Service, no expuesta a otros servicios de negocio. **Entrada:** `archivoId`, `version`, `nodoDestino`. **Salida:** confirmación de replicación + checksum verificado en destino. **Error:** `STG-601` (nodo destino no disponible — se reintenta con otro nodo según política de replicación). **Regla de negocio:** factor de replicación mínimo configurable (ej. 2 copias); si tras varios intentos no se alcanza el mínimo, se emite alerta a Monitoring, no se bloquea la escritura original (replicación es asíncrona respecto a `SubirArchivo`).

---

4. Modelo de datos

Archivo

Atributo	Tipo	Descripción	Restricciones
`id`	UUID	Identificador único, inmutable	PK
`rutaLogica`	string	Ruta dentro del árbol del usuario/compartido	Única dentro de su directorio padre
`propietarioId`	UUID	Usuario propietario	FK a Usuario (referencia lógica a Authentication)
`tipoMime`	string	Tipo de contenido	—
`versionActual`	int	Número de la última versión viva	⩾ 1
`eliminado`	boolean	Soft delete	—

Version

Atributo	Tipo	Descripción
`archivoId`	UUID	Archivo al que pertenece
`numeroVersion`	int	Secuencial, inmutable
`tamanoBytes`	long	—
`checksum`	string	Hash de integridad
`fechaCreacion`	datetime	—
`autorId`	UUID	Quién generó esta versión
`ubicacionesReplica`	array<string>	Nodos físicos donde existe copia

Directorio

Atributo	Tipo	Descripción
`id`	UUID	Identificador
`rutaLogica`	string	—
`padreId`	UUID (nullable)	Directorio contenedor
`permisos`	PermisoUnix	Owner/group/other (lectura/escritura/ejecución)

Cuota

Atributo	Tipo	Descripción
`usuarioId`	UUID	—
`espacioUsadoBytes`	long	Calculado sobre versiones vivas
`espacioLimiteBytes`	long	Límite asignado

PermisoUnix

Atributo	Tipo	Descripción
`propietario`	string (rwx)	—
`grupo`	string (rwx)	—
`otros`	string (rwx)	—

NodoAlmacenamiento

Atributo	Tipo	Descripción
`id`	string	Identificador de nodo físico
`capacidadTotal`	long	—
`capacidadDisponible`	long	—
`estado`	enum	`ACTIVO`, `DEGRADADO`, `NO_DISPONIBLE`

5. Eventos que genera el servicio

Evento	Disparado cuando	Consumidores típicos
`ArchivoSubido`	`SubirArchivo` exitoso	File Sync Service (para propagar a otros dispositivos), Photo Album Service (si es imagen, para indexar)
`ArchivoEliminado`	`EliminarArchivo`	File Sync Service, Shared File Service
`CuotaExcedida`	Intento de subida que supera el límite	Monitoring Service (alertas), Cliente (notificación al usuario)
`ReplicacionFallida`	`ReplicarArchivo` no alcanza el factor mínimo	Monitoring Service
`IntegridadComprometida`	`VerificarIntegridad` detecta checksum inconsistente	Monitoring Service (alerta crítica)
`HomeCreado`	`CrearHomeUsuario` exitoso	Shared File Service, Photo Album Service (pueden inicializar estructuras propias)

---

6. Integración

- **Quién consume:** Shared File Service, File Sync Service, Photo Album Service, Streaming Service y HPC Service (para guardar resultados) — todos a través del mismo contrato HTTP/REST.
- **Quién responde:** únicamente Distributed Storage Service; delega la validación de identidad en Authentication Service pero no delega ninguna operación de almacenamiento.
- **Protocolo:** HTTP/REST para todas las operaciones descritas.
- **Momento del flujo en que participa:** siempre después de que Authentication ya validó al usuario; es el paso final de escritura/lectura en prácticamente todos los flujos de negocio del sistema (HPC, Sync, Sharing, Photos, Streaming).

---

7. Consideraciones de escalabilidad

- **Qué puede crecer:** `SubirArchivo`/`DescargarArchivo` son las operaciones de mayor volumen de datos (no solo de solicitudes); el cuello de botella real suele ser ancho de

banda e I/O de disco, no CPU.

- **Cuellos de botella posibles:** el índice de metadatos, si es centralizado, se satura antes que el almacenamiento físico distribuido — conviene diseñar el metadato para poder partirse por rango de usuario/directorio.
- **Mejoras futuras:** replicación con selección de nodo por cercanía de red; cache de lectura para archivos "calientes" (los más descargados, típicamente resultados HPC recientes o contenido multimedia popular).

---

8. Casos límite

Operación	Caso límite	Comportamiento esperado
`SubirArchivo`	Dos clientes suben al mismo `rutaDestino` simultáneamente	Ambas se aceptan como versiones distintas; nunca se pierde una escritura silenciosamente
`SubirArchivo`	Corte de red a mitad de la transferencia	El archivo parcial nunca queda como versión válida/visible — se descarta y el cliente debe reintentar
`DescargarArchivo`	Todas las réplicas de un archivo no disponibles simultáneamente	Error explícito `STG-104` con detalle, nunca un timeout silencioso
`EliminarArchivo`	Se elimina un archivo mientras File Sync Service lo está descargando	La descarga en curso se completa (no se interrumpe), pero el archivo queda marcado como eliminado para futuras consultas
`RestaurarVersion`	Se restaura una versión mientras hay una subida concurrente de una versión nueva	Ambas operaciones generan versiones nuevas con números secuenciales distintos — nunca colisionan
`ActualizarCuota`	Se reduce la cuota por debajo del uso actual	Se permite (para casos administrativos), pero bloquea nuevas subidas hasta que el uso baje del nuevo límite

---

Contrato listo para implementación independiente de Distributed Storage Service. Siguiente en la secuencia: Shared File Service.